Guía de Laboratorio: Implementación de Arquitectura 2FA (TOTP) desde Cero y Observabilidad con Prefect

Curso: Patrones Arquitectónicos Avanzados
Modalidad: Laboratorio Práctico Controlado (Prueba de Concepto)
Tiempo estimado: 1 a 2 semanas

---

1. Contexto y Objetivo del Laboratorio

En la industria, delegar el doble factor de autenticación (2FA) a librerías caja negra oculta las decisiones arquitectónicas reales: cómo gestionar estados intermedios de autenticación, cómo evitar ataques de repetición (Replay Attacks) en ventanas de tiempo discretas y cómo observar qué ocurre dentro del pipeline de seguridad.

En este laboratorio construirás una Prueba de Concepto (PoC) controlada compuesta por dos piezas que se sincronizarán sin estar conectadas entre sí:

1. **Tu propia aplicación móvil Authenticator** (en React Native, SwiftUI o Android Studio/Kotlin) capaz de calcular códigos de 6 dígitos cada 30 segundos en modo avión.
2. **Base de datos con usuarios (supabase)**
3. **Sistema de login** - Web app.
4. **Un Backend de Autenticación Escalonada (Step-Up Auth)** cuyos flujos de aprovisionamiento y validación estarán instrumentados y orquestados mediante **Prefect** para visualizar cada paso en tiempo real.

Nota de alcance académico: Este es un entorno de laboratorio controlado. No necesitas desplegar en la nube, ni configurar clusters de Redis o bases de datos en producción; puedes usar SQLite o estructuras en memoria local (dict) y el servidor local de Prefect (prefect server start).

---

2. Arquitectura y Patrones a Implementar

Flujo General del Sistema

1. **Usuario Web / Cliente API:** Inicia sesión con usuario y contraseña en el Backend y recibe un `pre_auth_token` temporal.
2. **App Móvil Propia (Modo Avión):** Calcula `HMAC-SHA1(Secreto, UnixTime // 30)` cada 30 segundos y muestra el código de 6 dígitos.
3. **API Backend (Python/FastAPI):** Recibe el código de 6 dígitos junto con el `pre_auth_token` y ejecuta el Flow en Prefect.
4. **Prefect UI Dashboard:** Orquesta y monitorea las tareas:
    - **Task 1:** Validar Token Temporal (`validate_pre_auth_state`)
    - **Task 2:** Chequear Caché Anti-Replay (`check_anti_replay_cache`)
    - **Task 3:** Evaluar Strategy TOTP o Backup (`execute_mfa_strategy`)
    - **Task 4:** Emitir Access Token Final (`issue_elevated_session`)

Patrones Arquitectónicos Requeridos

- **Process Manager / Máquina de Estados (Aprovisionamiento Seguro):**
    - Cuando el usuario solicita activar 2FA, el estado de su configuración debe crearse como `PENDING_VERIFICATION`.
    - Solo cuando el usuario envía por primera vez un código válido generado desde su app móvil, el estado pasa a `ACTIVE`. (Esto evita dejar al usuario bloqueado si genera el QR pero nunca lo escanea).
- **Step-Up Authentication (Autenticación Escalonada):**
    - El endpoint `POST /auth/login` (usuario y contraseña) no entrega acceso a los recursos protegidos; devuelve un `pre_auth_token` con permisos restringidos únicamente para invocar `POST /auth/verify-2fa`.
    - Al superar `POST /auth/verify-2fa`, se entrega el `access_token` definitivo con el atributo `mfa_verified: true`.
- **Strategy Pattern:**
    - Crear una interfaz o clase base `SecondFactorStrategy` con dos implementaciones concretas intercambiables:
        - `TOTPStrategy`: Valida el código matemático de 6 dígitos (evaluando las ventanas de tiempo C-1, C y C+1).
        - `BackupCodeStrategy`: Valida códigos de emergencia de un solo uso por si el usuario pierde el teléfono.

---

3. Especificación Técnica Parte A: Tu Propia App Móvil

Los equipos pueden elegir libremente entre **React Native (Expo)**, **SwiftUI (iOS)** o **Android Studio (Kotlin/Jetpack Compose)**.

Requerimientos de la App

- **Prohibición de librerías TOTP hechas:** No se permite usar librerías que ya entreguen el código TOTP calculado (como `otplib`, `speakeasy` o equivalentes móviles). Sí se permite usar librerías nativas o estándar para calcular el hash criptográfico `HMAC-SHA1` y decodificar `Base32`.
- **Ingesta del Secreto:** La app debe permitir escanear el código QR generado por tu backend (extrayendo el parámetro `secret` de la URI `otpauth://totp/...`) o tener un campo de texto para pegar la semilla en Base32 manualmente (muy útil si trabajan en emulador/simulador sin cámara).

Algoritmo Paso a Paso que debe ejecutar tu App

1. Decodificar el `secret` de Base32 a un arreglo de bytes (K).
2. Obtener el Unix Timestamp actual en segundos y calcular el contador entero:

$$C = \lfloor \text{UnixTime} / 30 \rfloor$$

3. Empaquetar C como un entero de 8 bytes (64 bits, Big-Endian).
4. Calcular el hash $\text{HMAC-SHA1}(K, C)$, que producirá un arreglo de 20 bytes.
5. **Truncamiento Dinámico:**
    - Tomar el último byte del hash (índice 19) y aplicar una máscara para obtener el offset:

      $$\text{offset} = \text{hash}[19] \ \& \ \text{0x0f}$$

    - Extraer 4 bytes consecutivos empezando en `offset`: `hash[offset ... offset + 3]`.
    - Quitar el bit de signo al primer byte (`& 0x7f`) para obtener un entero positivo de 31 bits.
    - Aplicar módulo 1,000,000 (`valor % 1000000`) y rellenar con ceros a la izquierda si tiene menos de 6 dígitos.

Interfaz de Usuario (UI)

- Mostrar el código de 6 dígitos en tamaño grande.

- Mostrar una barra de progreso o contador regresivo de 30 a 0 segundos que regenere el código automáticamente al llegar a 0.

Tip de Verificación Cruzada: Como están implementando el estándar RFC 6238 real, si escanean el mismo código QR con su propia app y al mismo tiempo con Google Authenticator o Microsoft Authenticator reales, ambas aplicaciones deben mostrar exactamente los mismos 6 dígitos al mismo tiempo.

---

4. Especificación Técnica Parte B: Backend + Observabilidad con Prefect

Para facilitar la integración con **Prefect**, se recomienda construir el backend en **Python (FastAPI o Flask)**.

Flujos (@flow) y Tareas (@task) en Prefect

Cada vez que un estudiante intente iniciar sesión o verificar su código 2FA, el endpoint de la API debe invocar un @flow de Prefect compuesto por las siguientes tareas observables:

1. `@task validate_pre_auth_state`: Verifica que el usuario venga con un token temporal válido.
2. `@task check_anti_replay_cache`: Verifica en un diccionario o caché en memoria si la combinación `(user_id, totp_code)` ya fue usada en los últimos 90 segundos. Si ya se usó, lanza una excepción clara en Prefect (`ReplayAttackDetected`).
3. `@task execute_mfa_strategy`: Ejecuta la estrategia seleccionada (`TOTPStrategy` o `BackupCodeStrategy`). En el caso de TOTP, registra mediante `get_run_logger()` de Prefect en cuál de las 3 ventanas (C-1, C o C+1) hizo match el código (permite observar en vivo si el reloj del celular del estudiante está atrasado o adelantado).
4. `@task issue_elevated_session`: Marca el código como usado en la caché Anti-Replay y genera el JWT final con `mfa_verified: true`.

---

5. Pruebas de Aceptación (Demo en el Laboratorio)

Durante la sustentación en clase, cada grupo realizará las siguientes 4 pruebas rápidas en vivo (5 minutos por grupo):

1. **Prueba de Paridad Estándar:** Escanear el QR del backend tanto en la App móvil creada por el equipo como en Google/Microsoft Authenticator oficial. Demostrar que ambas apps generan el mismo código simultáneamente.
2. **Prueba de Modo Avión:** Activar el modo avión en el celular o emulador, esperar a que cambie la ventana de 30 segundos, e iniciar sesión exitosamente en el backend consumiendo una ruta protegida (`GET /api/boveda-secreta`).
3. **Prueba de Ataque de Repetición (Anti-Replay):** Copiar el mismo código de 6 dígitos recién utilizado e intentar enviarlo una segunda vez antes de que se acaben los 30 segundos. El backend debe rechazarlo y en el Dashboard de Prefect debe verse el Flow marcado en rojo (`Failed`) exactamente en la tarea `check_anti_replay_cache`.
4. **Prueba de Desfase de Reloj (Clock Skew) o Código de Respaldo:** Demostrar en los logs de la interfaz de Prefect qué ventana (C-1, C, C+1) validó el código, o autenticar usando un código de emergencia (`BackupCodeStrategy`).

---

6. Entregables y Rúbrica de Evaluación

Entregables

- Repositorio de Git con dos carpetas claras: `/mobile-authenticator` y `/backend-prefect`.
- Archivo `README.md` con instrucciones para levantar el servidor de Prefect (`prefect server start`), el backend y la app móvil.

Rúbrica de Calificación (Sobre 5.0 / 100%)

Criterio	Peso	Descripción de Cumplimiento
Implementación Criptográfica TOTP en App Móvil	30%	La app móvil (React Native, SwiftUI o Kotlin) calcula el código de 6 dígitos desde cero (HMAC-SHA1 + Truncamiento Dinámico) cada 30s sin conexión y coincide con Google/Microsoft Authenticator.
Patrones Arquitectónicos (Step-Up, Strategy, Estado)	30%	El backend implementa correctamente el estado `PENDING_VERIFICATION`, la

Criterio	Peso	Descripción de Cumplimiento
		separación entre Pre-Auth Token y Access Token, y el patrón Strategy para el segundo factor.
Idempotencia y Prevención de Replay Attacks	15%	Un código válido de 6 dígitos no puede ser utilizado dos veces dentro de la misma ventana de validez de tiempo.
Observabilidad y Orquestación con Prefect	25%	El pipeline de verificación está decorado con `@flow` y `@task` en Prefect, permitiendo visualizar en la UI el grafo de ejecución, los bloqueos por Replay Attack y los logs de ventana temporal.